Mastering TypeScript

Comprehensive Practice Tasks: From Basics to Advanced Types

This document contains a curated set of tasks designed to bridge the gap between theoretical knowledge and practical application. Each task represents a real-world scenario you will likely encounter in professional web development.

Task 1: The "Optional" Shopping Cart

EASY

Concepts: Destructuring, Optional Properties, Default Values

Scenario: You are building a checkout system. Users might buy one item by default, or specify a bulk quantity.

```
type CartItem = {
    name: string;
    price: number;
    quantity?: number;
};
```

Instructions:

- Write a function `calculateTotal` that takes a `CartItem` object.
- Use **Destructuring** to extract properties.
- If `quantity` is missing, ensure the calculation treats it as `1`.
- Return the total cost (`price * quantity`).

> **Hint:** Set a default value during destructuring: `{ quantity = 1 } = item`.

Task 2: Merging User Profiles

EASY

Concepts: Intersection Types (&)

Scenario: A user signs up as a basic `Person`, but when hired, they gain `JobDetails`. An `Employee` is a union of both.

```
type Person = { name: string; age: number };
type JobDetails = { role: string; salary: number };
```

Instructions:

- Create a new type `Employee` that combines `Person` and `JobDetails`.
- Write a function `getProfile` that accepts an `Employee`.
- Return a string: `"Name: [name], Role: [role]"`.

> **Hint:** Use the & operator to merge the two types.

Task 3: The "Safe" Data Fetcher

MEDIUM

Concepts: Optional Chaining (?.), Nullish Coalescing (??)

Scenario: API responses can be unpredictable. You need to safely access a deep property without causing a crash.

```
type UserResponse = {
    info?: {
        address?: {
            zipCode?: string;
        }
    }
};
```

Instructions:

- Write a function `getZipCode` that reaches deep into the object.
- If any part of the path is missing, or if the zipCode is null/undefined, return `"00000"`.

> **Hint:** Chain `?.` for every level and end with `?? "00000"`.

Task 4: Type Assertion

MEDIUM

Concepts: Type Assertion (as), unknown type

Scenario: You receive a value from a 3rd-party library typed as `unknown`. You are certain it's a string and need to manipulate it.

```
let secretValue: unknown = "typescript is awesome";
```

Instructions:

- Create a variable `upperValue`.
- Assign `secretValue` to it using Type Assertion.
- Call `.toUpperCase()` on the resulting value.

> **Hint:** Use the `value as string` syntax.

Task 5: Generic Constraints

MEDIUM

Concepts: Generics, Extends Constraint

Scenario: You want a function that logs the length of various inputs (strings, arrays) but rejects types that don't have a `.length`.

Instructions:

- Write a generic function `logLength<T>(input: T)`.
- Constrain `T` to ensure it must have a `length` property of type `number`.
- Return the length value.

> **Hint:** Use `<T extends { length: number }>`.

Task 6: The Property Guard

HARD

Concepts: keyof, Generics

Scenario: Create a utility that gets a property from an object while preventing typos at compile-time.

```
const product = { id: 101, name: "Keyboard", price: 50 };
```

Instructions:

- Create a function `getProductProp<T, K>(obj: T, key: K)`.
- Constraint `K` to be a valid key of `T`.
- Return `obj[key]`.

> **Hint:** Use `<T, K extends keyof T>`.

Task 7: Constant Literal Types

HARD

Concepts: as const, typeof, Index Access Types

Scenario: Define fixed theme colors that serve as the single source of truth for your application.

```
const Colors = {
    Primary: "RED",
    Secondary: "BLUE"
} as const;
```

Instructions:

- Create a type `ValidColor` derived directly from the values of the `Colors` object.
- Write a function `setColor(c: ValidColor)` that only accepts `"RED"` or `"BLUE"`.

> **Hint:** `type ValidColor = typeof Colors[keyof typeof Colors];`.

Task 8: The "Draft" Mode

HARD

Concepts: Mapped Types, Readonly, Optional

Scenario: Transform a strict interface into a "Draft" version where everything is optional and immutable.

```
interface MyDocument {
    title: string;
    content: string;
    author: string;
}
```

Instructions:

- Create a Mapped Type `Draft<T>`.
- Iterate through all keys of `T`, making them `readonly` and `?` (optional).
- Declare a variable `myDraft` of type `Draft<MyDocument>`.

> **Hint:** `{ readonly [P in keyof T]?: T[P] }`.

Task 9: The Wrapper

HARD

Concepts: Conditional Types

Scenario: Create a type that acts as a logic gate, returning "Large" for arrays and "Small" for anything else.

Instructions:

- Create a type `DataType<T>`.
- If `T` extends an array, the type should be `"Large"`.
- Otherwise, it should be `"Small"`.

> **Hint:** Use the ternary syntax: `T extends any[] ? "Large" : "Small"`.

Task 10: Utility Type (Omit)

MEDIUM

Concepts: Built-in Utility Types (Omit)

Scenario: You need to strip sensitive data (like a password) from a user object before sending it to the UI.

```
interface UserAccount {
    id: number;
    username: string;
    password: string;
}
```

Instructions:

- Create a type `PublicUser` using the `Omit` utility.
- Exclude the `password` field from `UserAccount`.

> **Hint:** `Omit<UserAccount, "password">`.

Solution Key

Task 1: Optional Shopping Cart

```
const calculateTotal = ({ price, quantity = 1 }: CartItem): number => {
    return price * quantity;
};
```

Task 2: Merging User Profiles

```
type Employee = Person & JobDetails;

function getProfile(emp: Employee): string {
    return `Name: ${emp.name}, Role: ${emp.role}`;
}
```

Task 3: Safe Data Fetcher

```
function getZipCode(response: UserResponse): string {
    return response.info?.address?.zipCode ?? "00000";
}
```

Task 4: Type Assertion

```
let secretValue: unknown = "typescript is awesome";
const upperValue = (secretValue as string).toUpperCase();
```

Task 5: Generic Constraints

```
function logLength<T extends { length: number }>(input: T): number {
    return input.length;
}
```

Task 6: The Property Guard

```
function getProductProp<T, K extends keyof T>(obj: T, key: K) {
    return obj[key];
}
// Usage: getProductProp(product, "price"); // 50
```

Task 7: Constant Literal Types

```
const Colors = { Primary: "RED", Secondary: "BLUE" } as const;
type ValidColor = typeof Colors[keyof typeof Colors];

function setColor(c: ValidColor) {
    console.log(`Setting color to ${c}`);
}
```

Task 8: The "Draft" Mode

```
type Draft<T> = {
    readonly [P in keyof T]?: T[P];
};

const myDraft: Draft<MyDocument> = { title: "Draft 1" };
```

Task 9: The Wrapper

```
type DataType<T> = T extends any[] ? "Large" : "Small";

type Test1 = DataType<string[]>; // "Large"
type Test2 = DataType<number>;   // "Small"
```

Task 10: Utility Type (Omit)

```
type PublicUser = Omit<UserAccount, "password">;
```